Building a Chatbot with UI in LangGraph & Streamlit

1. Introduction

- The goal is to build a **chatbot application with a user interface**.
- **LangGraph** handles the chatbot's workflow and state.
- **Streamlit** provides the web-based chat interface.

Architecture:

User → Streamlit UI → LangGraph → LLM → Response → Streamlit UI

2. Chatbot Demo

- The completed application works like a normal chat interface.
- The user can:
  - Enter a message.
  - Receive an AI response.
  - Continue the conversation.
- LangGraph manages the conversation workflow behind the UI.

3. Plan of Action

The chatbot is built in several stages:

1. Set up the project.
2. Create the LangGraph chatbot workflow.
3. Connect the LLM.
4. Add persistence/conversation state.
5. Build the Streamlit chat interface.
6. Connect the UI with the LangGraph workflow.
7. Test the complete application.

4. Code Setup

- Set up the Python project and required dependencies.
- Configure the LLM and LangGraph.
- Create the chatbot graph.
- Define the required **state and nodes**.
- Compile the graph so it can be executed by the application.

The backend is responsible for **processing messages and generating responses**.

5. Chat Interface in Streamlit

Streamlit is used to create the chatbot's web UI without building a frontend from scratch.
The interface generally contains:

- Chat message history

- User input box
- User messages
- AI responses
- Chat session/state handling

Basic flow:

User enters message → Streamlit receives it → LangGraph processes it → AI response → Streamlit displays it

The UI handles **interaction**, while LangGraph handles the **AI workflow**.

6. Final Demo

The final application combines everything:

Streamlit UI
↓
User Message
↓
LangGraph Workflow
↓
LLM
↓
Response
↓
Streamlit Chat UI

Key takeaway: Streamlit provides the **chat interface**, while LangGraph manages the **chatbot's workflow and state**. Together, they can be used to build a functional AI chatbot application.